Sri Lanka Institute of Information Technology

B.Sc. (Hons) Information Technology

Reverse Engineering and Controlled Malware Injection in Android Applications (Educational Trojan Simulation)

Year 03 Semester 01

IE3112 – Mobile Security

Table of Contents

1. Reverse Engineering

1.1. APK Structure Analysis

We used MobSF (Mobile Security Framework) v4.5.0 and apktool v3.0.2 to look at the HabitBook APK (version 1.0.14). The app got a security score of 51 out of 100 (Medium Risk) and a grade of B, which means there are a lot of holes that hackers could use.

File Name	Habitbook.apk
Package Name	com.gethabitbook.app
App Version	1.0.14 (Version Code: 56)
File Size	13.08 MB
MD5 Hash	5460d1bff3da8f4e8ea2a5846e9a1f11
SHA256 Hash	0ca4a40a1045956dbe85fc3f60d731ff3779098aa223ff5e671a61beebe629eb
Target SDK	35 (Android 15)
Minimum SDK	26 (Android 8.0)
Security Score	51/100 – MEDIUM RISK (Grade: B)
Trackers Detected	2/432 (Branch Analytics, OneSignal)
Compiler	R8 (with Anti-VM checks detected)

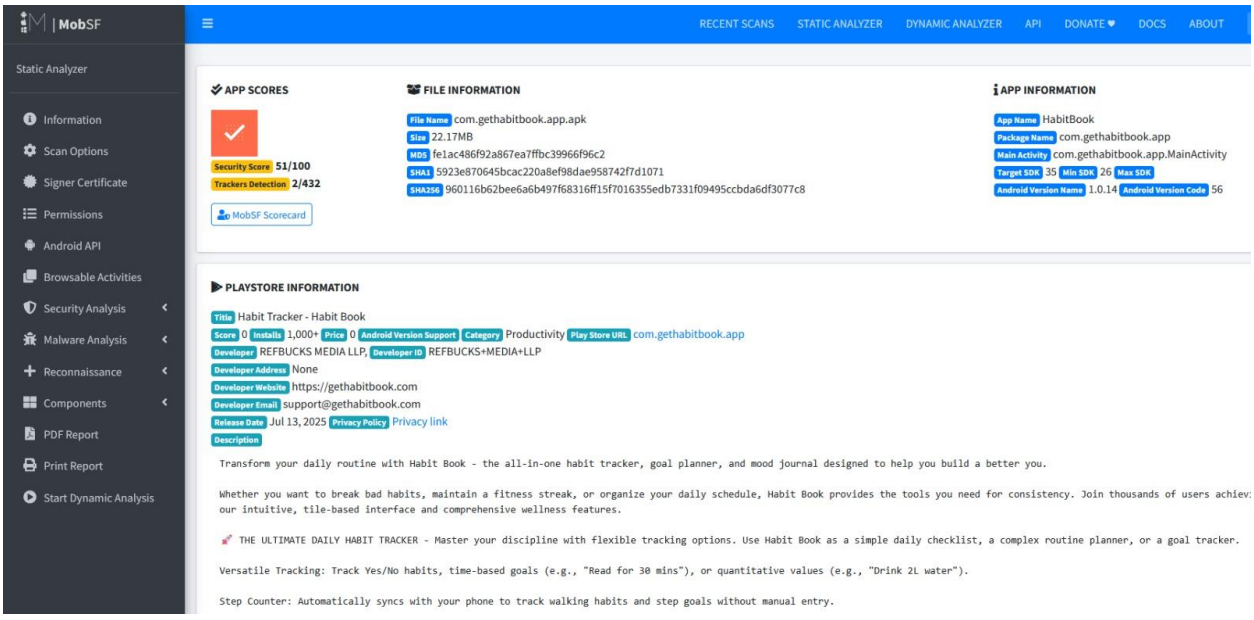


Figure 1: MobSF dashboard showing the security score, app info, and grade B here

The APK was decompiled using apktool, which extracted the following directory structure:

```
apktool d Habitbook.apk -o HabitBook_decompiled --force
```

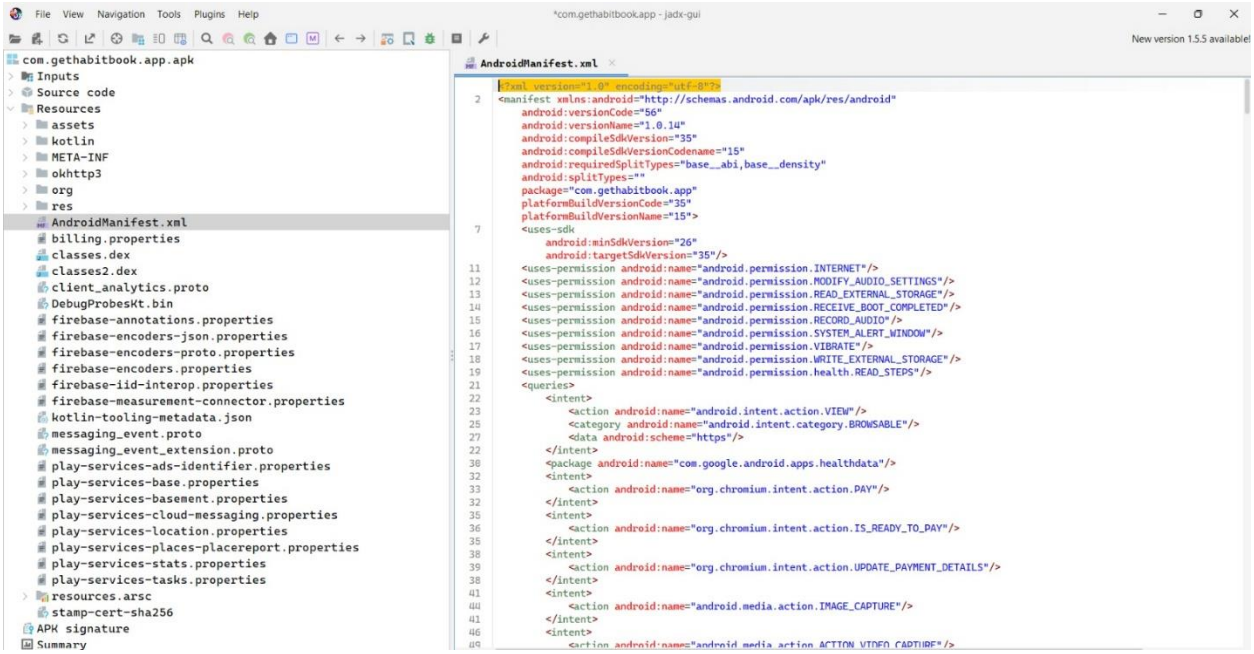


Figure 2: Decompiled folder structure showing AndroidManifest.xml, smali/, res/, assets/ folders

1.2. Application Component Mapping

The application has a lot of parts, and many of them can be used by other applications. This makes the application's attack surface much bigger.

Component	Total	Exported	Risk
Activities	14	**5**	High – accessible by other apps
Services	13	**2**	Medium – background execution
Broadcast Receivers	21	**8**	High – intercept broadcasts
Content Providers	8	**0**	Low – not exported

Key exported components that create security risks include:

- com.gethabitbook.app.MainActivity – The main entry point that can be exported and accessed.
- androidx.health.platform.client.impl.sdkservice.HealthDataSdkService – An exported service that isn't protected.
- com.canhub.cropper.CropImageActivity – An exported activity that isn't protected.
- com.onesignal.notifications.receivers.BootUpReceiver – A receiver that starts up when the computer boots up.
- com.onesignal.notifications.receivers.FCMBroadcastReceiver – Protected by permission, but the level of protection is not known.

1.3. Permission Analysis

The application requests more than 35 permissions, six of which are dangerous and could put your privacy at risk. The MobSF analysis discovered that 10 out of 25 permissions correspond to those frequently exploited by recognised malware.

Permission	Status	Risk
CAMERA	**dangerous**	Allows capturing images/video without user awareness
RECORD_AUDIO	**dangerous**	Enables microphone access for audio recording
READ_EXTERNAL_STORAGE	**dangerous**	Reads all files on external storage
WRITE_EXTERNAL_STORAGE	**dangerous**	Writes/modifies/deletes external storage files
SYSTEM_ALERT_WINDOW	**dangerous**	Draws overlays over other apps – abused by spyware
ACTIVITY_RECOGNITION	**dangerous**	Tracks physical movement of the user
POST_NOTIFICATIONS	**dangerous**	Sends push notifications without consent
RECEIVE_BOOT_COMPLETED	**normal**	Auto-starts on device boot – persistence mechanism
INTERNET	**normal**	Unrestricted internet access – enables data exfiltration
ACCESS_NETWORK_STATE	**normal**	Monitors network connectivity changes

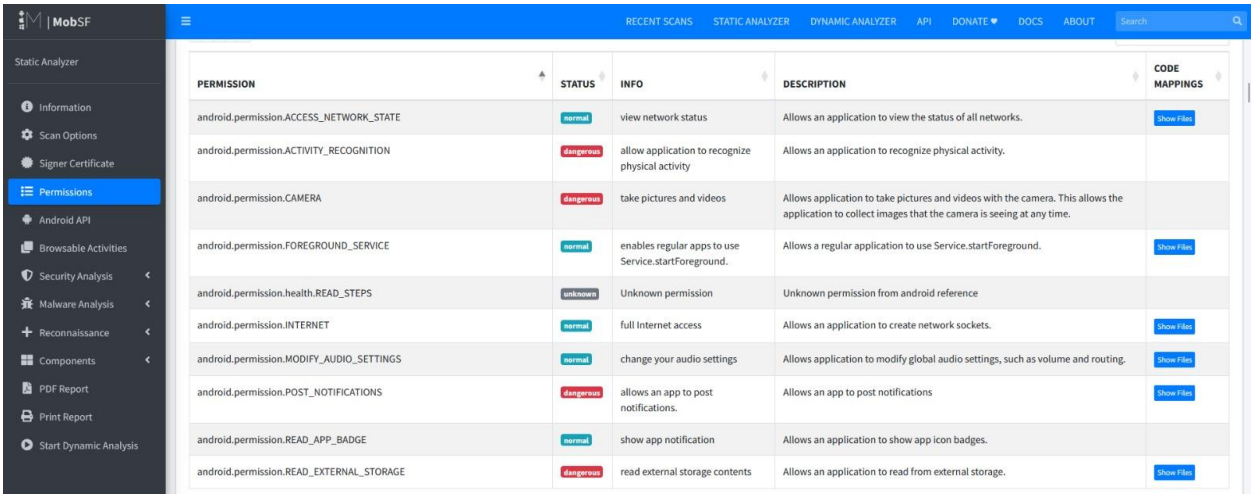


Figure 3: MobSF permissions table showing dangerous permissions highlighted in red

It's important to note that SYSTEM_ALERT_WINDOW was already declared in the first application. This permission, along with the fact that the app already has access to the INTERNET and CAMERA, makes it a prime target for bad changes.

1.4. Control Flow Analysis

We used jadx to decompile the APK into readable Java so we could look at how the app worked. The main entry point is MainActivity, which is an extension of the React Native activity class com.facebook.react.p. The app's user interface is built on the React Native framework, and background tasks are handled by native Android services.

MainActivity

```
1  package com.gethabitbook.app;
2
3  import S8.m;
4  import android.os.Build;
5  import android.os.Bundle;
6  import com.facebook.react.AbstractActivityC1516p;
7  import com.facebook.react.C1543s;
8  import e4.i;
9  import kotlin.Metadata;
10
11 @Metadata(d1 = {"\u0000*\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u0002\b\u0002\n\u0002\u0018\u0002\n\u0000\n\u0002\u0018\u0002\n\u0002\b\u0002\
12 /* loaded from: classes.dex */
13 public final class MainActivity extends AbstractActivityC1516p {
14
15     public static final class a extends com.facebook.react.defaults.b {
16         a(MainActivity mainActivity, String str, boolean z10) {
17             super(mainActivity, str, z10);
18         }
19     }
20
21     @Override // com.facebook.react.AbstractActivityC1516p
22     protected C1543s F0() {
23         return new m(this, false, new a(this, H0(), com.facebook.react.defaults.a.a()));
24     }
25
26     protected String H0() {
27         return "main";
28     }
29
30     @Override // com.facebook.react.AbstractActivityC1516p, B3.a
31     public void c() {
32         if (Build.VERSION.SDK_INT > 30) {
33             super.c();
34         } else {
35             if (moveTaskToBack(false)) {
36                 return;
37             }
38             super.c();
39         }
40     }
41
42     @Override // com.facebook.react.AbstractActivityC1516p, androidx.fragment.app.AbstractActivityC1234s, androidx.activity.ComponentActivity, ar
43     protected void onCreate(Bundle savedInstanceState) {
44         setTheme(i.f26030a);
45         super.onCreate(null);
46     }
47 }
```

Figure 4: jadx showing decompiled MainActivity Java code

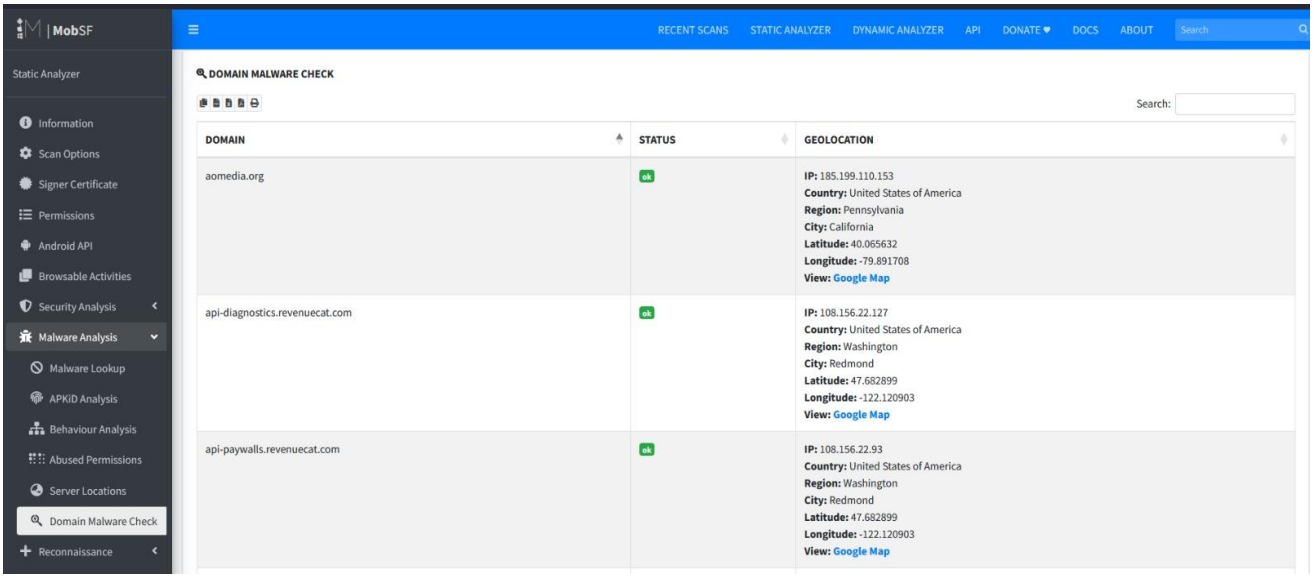
Application startup flow:

- System launches com.gethabitbook.app.MainActivity on app start.
- MainActivity calls super.onCreate() which initializes the React Native runtime.
- React Native loads the JavaScript bundle from assets.
- Background services are initialized including OneSignal notification service.
- Widget receivers are registered for home screen widget updates.

1.5. Identified Vulnerabilities

Two HIGH severity, nine WARNING severity, and seventeen manifest warnings were found using MobSF static analysis. The important conclusions are summed up in the following table:

Severity	Issue	Description
HIGH	Remote WebView Debugging Enabled	The release build (MSTG-RESILIENCE-2) has WebView debugging enabled. enables JavaScript injection by attackers using Chrome DevTools.
HIGH	CBC/PKCS7 Padding Oracle Vulnerability	PKCS5/7-padded AES-CBC encryption is susceptible to padding oracle attacks (CWE-649, MSTG-CRYPTO-3).
WARNING	Hardcoded Secrets Detected	Nine potential secret keys or tokens were discovered hardcoded in the source code (CWE-312, MSTG-STORAGE-14).
WARNING	Weak MD5 Hash Algorithm	MD5 is used in multiple files – known to have hash collisions (CWE-327, MSTG-CRYPTO-4).
WARNING	Weak SHA-1 Hash Algorithm	SHA-1 is used in multiple files – deprecated and vulnerable (CWE-327, MSTG-CRYPTO-4).
WARNING	Insecure Random Number Generator	java.util.Random is used instead of SecureRandom (CWE-330, MSTG-CRYPTO-6).
WARNING	SQL Injection Risk	Raw SQL queries executed with untrusted input (CWE-89, MSTG-PLATFORM-7).
WARNING	External Storage Read/Write	Apps can read and write to external storage that other apps can access (CWE-276, MSTG-STORAGE-2).
WARNING	Insecure WebView Implementation	JavaScript execution in WebView without appropriate input validation (CWE-749).
WARNING	IP Address Disclosure	Hardcoded IP addresses found in source code (CWE-200, MSTG-CODE-2).
INFO	Sensitive Information Logged	App logs sensitive information to logcat (CWE-532, MSTG-STORAGE-3).



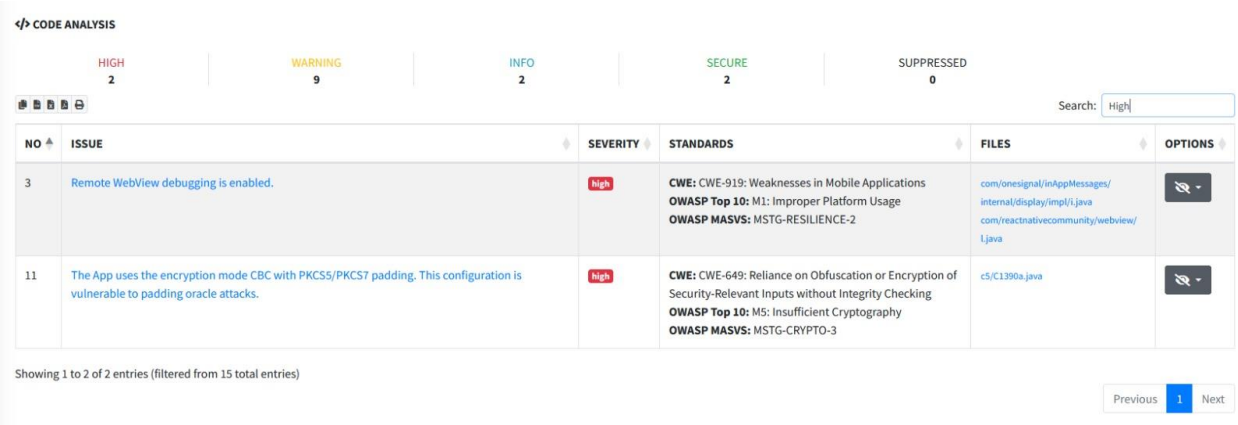


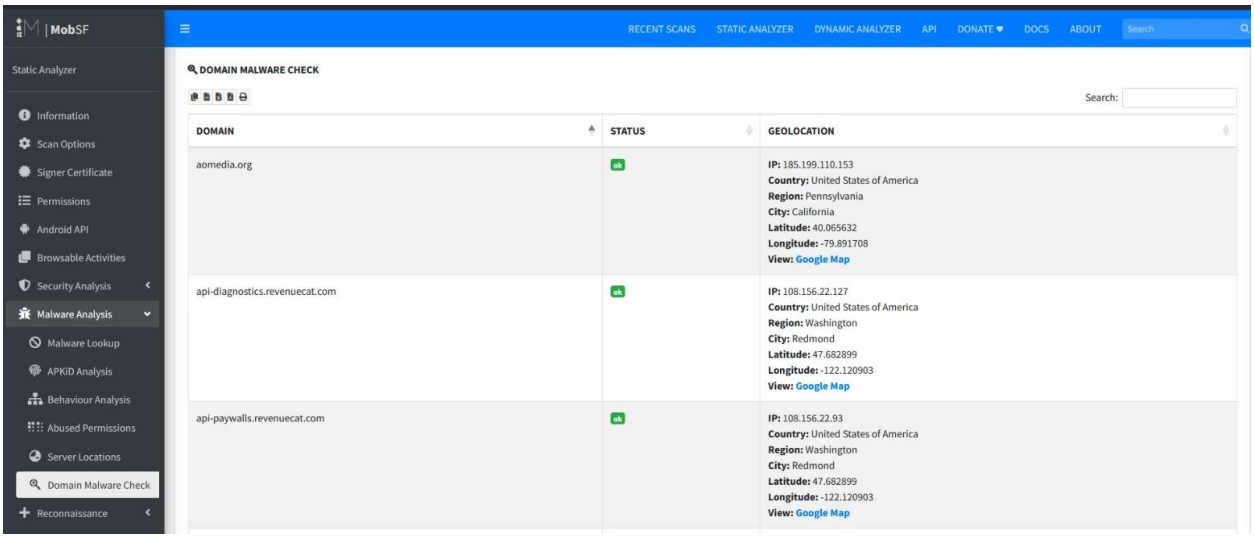
Figure 5: MobSF code analysis section showing HIGH severity findings

1.6. Network Analysis

The program interacts with several external areas. Malware databases were used to verify that every domain was clean. Two SDKs for tracking were found:

- Branch Analytics: This tool is used for attribution tracking and deep linking (https://reports.exodus-privacy.eu.org/trackers/167).
- OneSignal is used to deliver push notifications (https://reports.exodus-privacy.eu.org/trackers/193).

Key domains include api.revenuecat.com (in-app purchases), api.onesignal.com (notifications), and api2.branch.io (deep linking). All resolved to legitimate US-based servers.



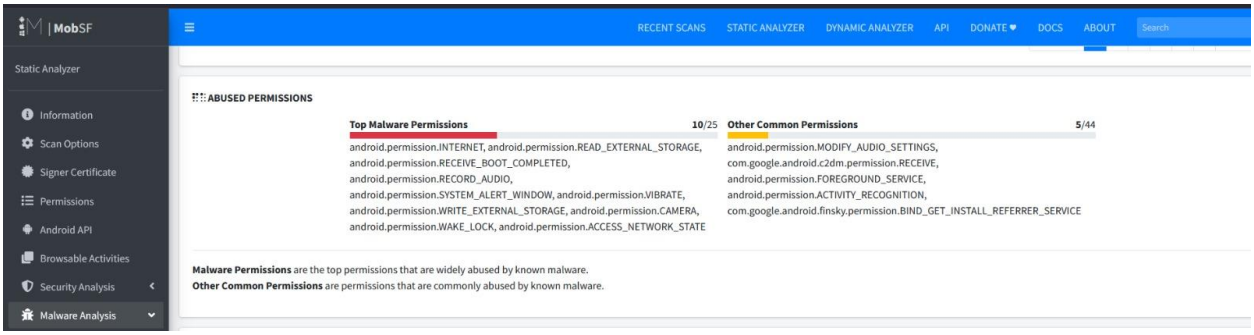


Figure 6: MobSF domain malware check section showing all domains with OK status

1.7. Certificate Analysis

A Google Debug certificate is used to sign the application (v2 and v3 signatures). This is odd for a production program and raises the possibility that the developer signed releases using a debug keystore, which raises security issues.

Signing Status	Signed (v2 + v3 signatures)
Certificate Subject	C=US, ST=California, O=Google Inc., CN=Android
Issuer	C=US, ST=California, O=Google Inc., CN=Android
Valid From	2025-07-09
Valid To	2055-07-09 (30 years)
Key Algorithm	RSA 4096-bit
Hash Algorithm	SHA-256

2. Malicious Service Injection

2.1. Injection Strategy

In order to mimic Trojan-like behavior, a background service called SyncService was injected. The purpose of the service was to:

- Operate in the background discreetly and without the user's knowledge.
- Only activate when a certain date and time are reached (trigger at 12:00 PM).
- To mimic data acquisition, log sensitive device information.
- Data exfiltration to a localhost server can be simulated.

The injection was less obvious because the original manifest already included the SYSTEM_ALERT_WINDOW permission. To guarantee that the service runs each time the application opens, it was started from MainActivity.onCreate().

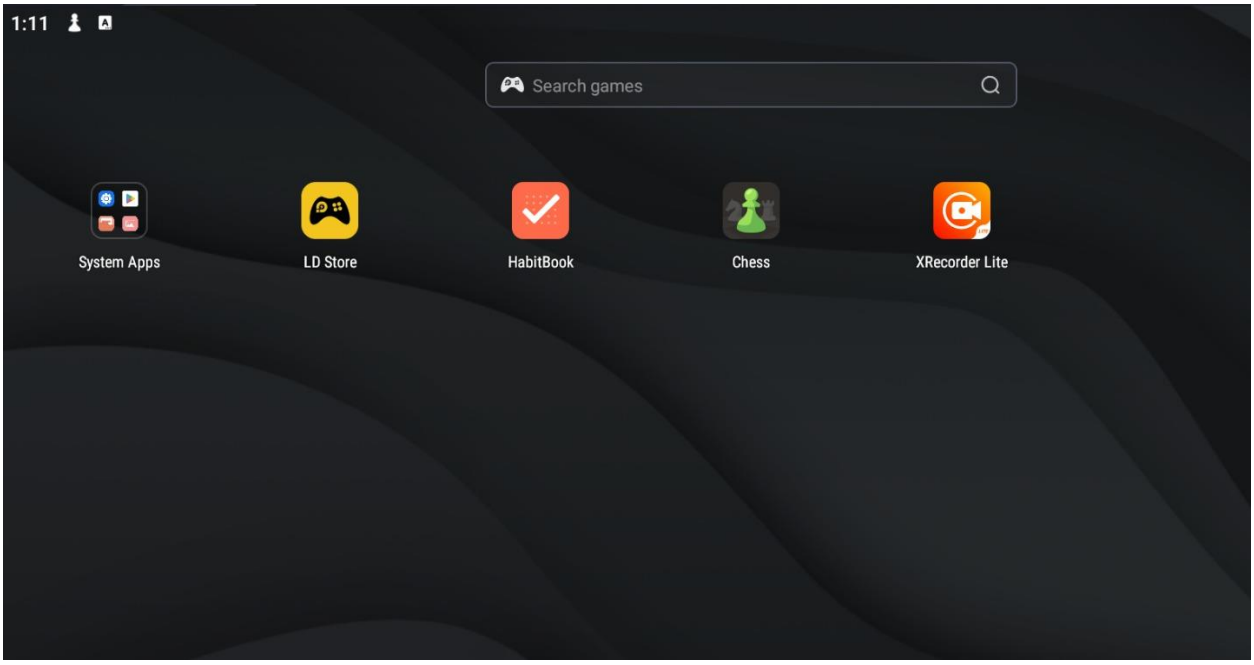


Figure 7: LDPlayer showing HabitBook app installed and running normally

2.2. Tools Used

Tool	Version	Purpose
apktool	3.0.2	Decompile and recompile APK
jadx	Latest	Decompile DEX to readable Java
MobSF	v4.5.0	Static analysis and vulnerability scanning
uber-apk-signer	1.3.0	Sign modified APK
ADB	Built-in	Install APK and collect logcat output
LDPlayer	9.1.63.3	Android 9 emulator for testing
Visual Studio Code	Latest	Edit smali and XML files

2.3. Step-by-Step Injection Process

Step 1 – Decompile the APK

The base.apk was extracted from the XAPK package and decompiled using apktool:

```
apktool d base.apk -o HabitBook_decompiled --force
```

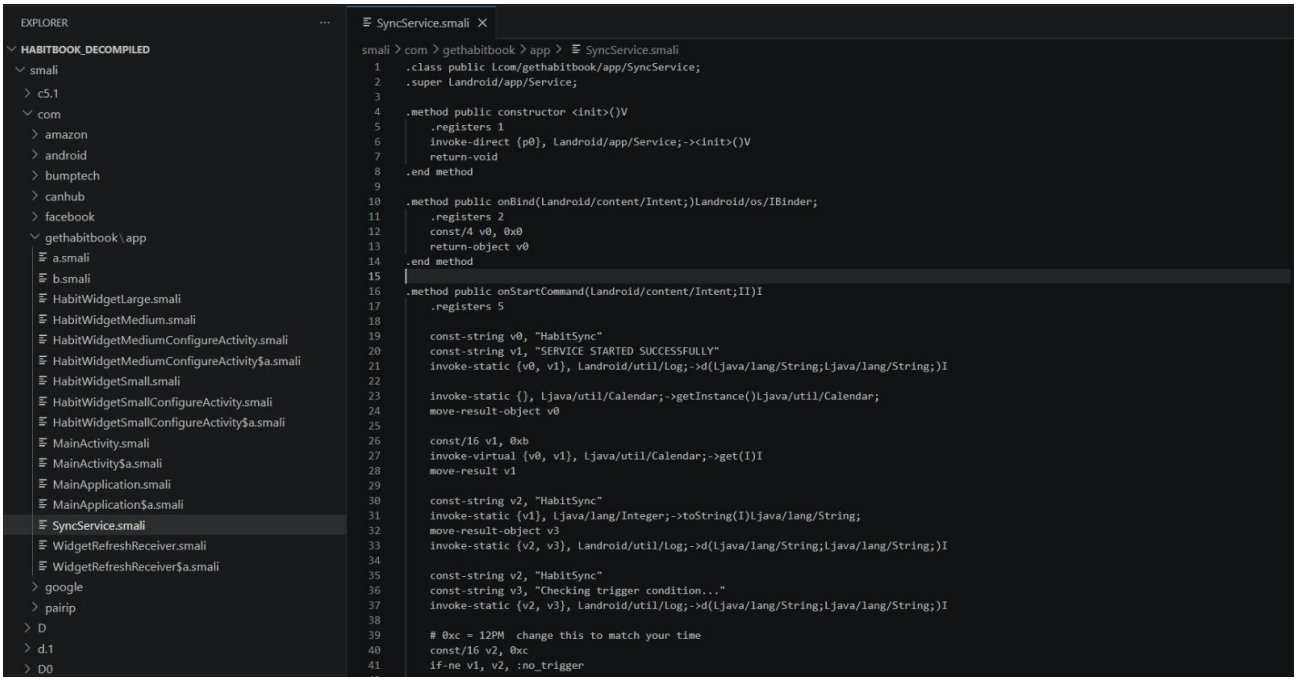
```
C:\Users\Admin\OneDrive\Desktop\Y3S1\MS\Assignment\Habit Tracker - Habit Book>apktool d com.gethabitbook.app.apk -o HabitBook_decompile --force
I: Using Apktool 3.0.2 on com.gethabitbook.app.apk with 4 threads
I: Loading resource table...
I: Baksmaling classes.dex...
I: Baksmaling classes2.dex...
I: Decoding value resources...
I: Loading resource table from file: C:\Users\Admin\AppData\Local\apktool\framework\1.apk
I: Decoding file resources...
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f080034, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f080033, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f08002e, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f08002d, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f080105, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f080072, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f080072, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f080071, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f080050, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f08004f, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f08015b, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f08015a, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f08015d, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f08015c, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f080055, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f080055, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f08004e, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f080054, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f080054, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f08004e, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f08005d, type=ref}
W: Unresolved resource reference: ResReference{pkg=ResPackage{id=0x7f, name=com.gethabitbook.app}, id=0x7f08005d, type=ref}
```

Figure 8: CMD showing apktool decompile output with 'I: Built apk' success message

Step 2 – Create SyncService.smali

A new smali file was created at HabitBook_decompiled\smali\com\gethabitbook\app\SyncService.smali. The service implements the following logic in onStartCommand():

- Logs service startup to logcat using Android Log.d().
- Gets current time using Calendar.getInstance().
- Checks if current hour matches trigger hour (12 = 12PM).
- If trigger fires: logs device info and simulates exfiltration.
- Returns START_STICKY to ensure service restarts if killed.



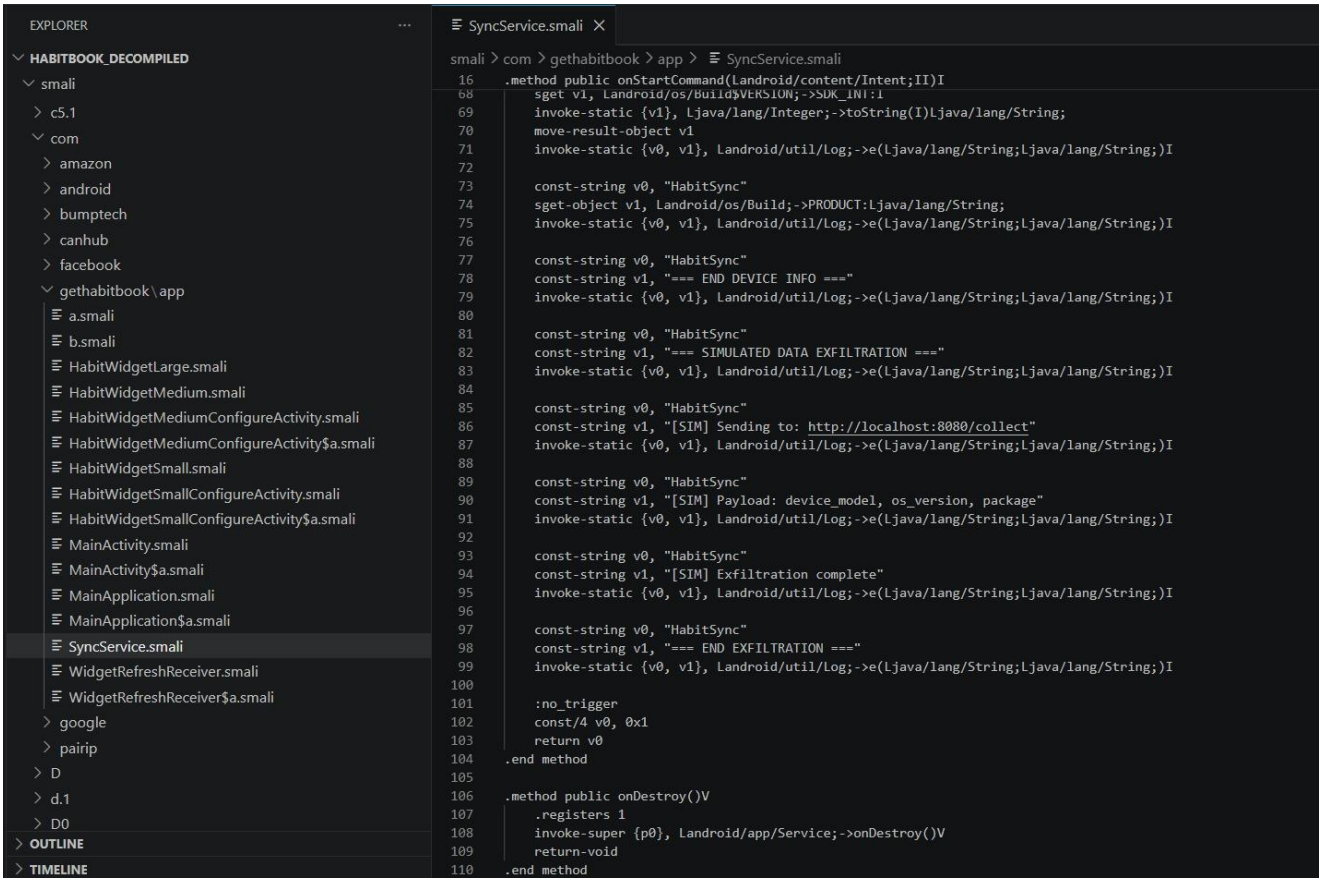


Figure 9: SyncService.smali file open in Notepad++ showing the full smali code

Step 3 – Modify AndroidManifest.xml

The service was registered in AndroidManifest.xml by adding the following tag before </application>:

```
<service android:name="com.gethabitbook.app.SyncService" android:enabled="true"
android:exported="false" />
```

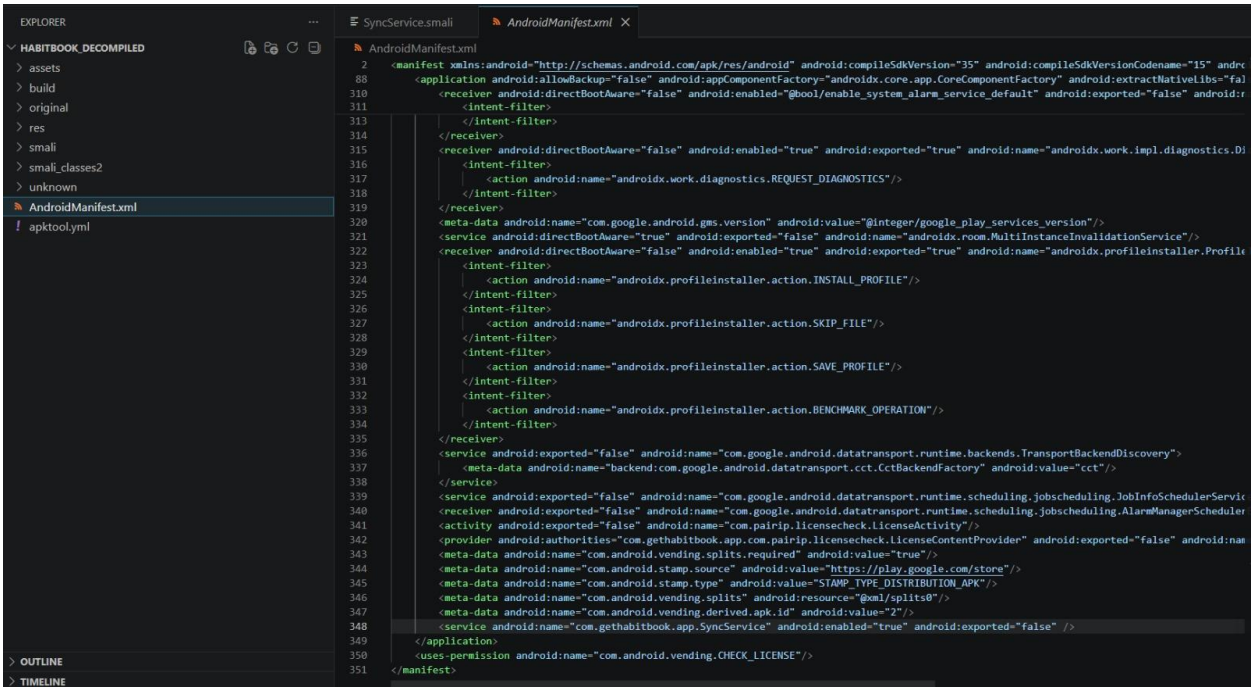


Figure 10: AndroidManifest.xml in Notepad++ showing the SyncService tag added before </application>

Step 4 – Inject Service Start into MainActivity.smali

The MainActivity.smali was modified to start the service on application launch. The .locals count was increased from 0 to 2 to accommodate the new registers:

```
.locals 2

new-instance v0, Landroid/content/Intent;

const-class v1, Lcom/gethabitbook/app/SyncService;

invoke-direct {v0, p0, v1}, Landroid/content/Intent;-
><init>(Landroid/content/Context;Ljava/lang/Class;)V

invoke-virtual {p0, v0}, Lcom/gethabitbook/app/MainActivity;-
>startService(Landroid/content/Intent;)Landroid/content/ComponentName;
```

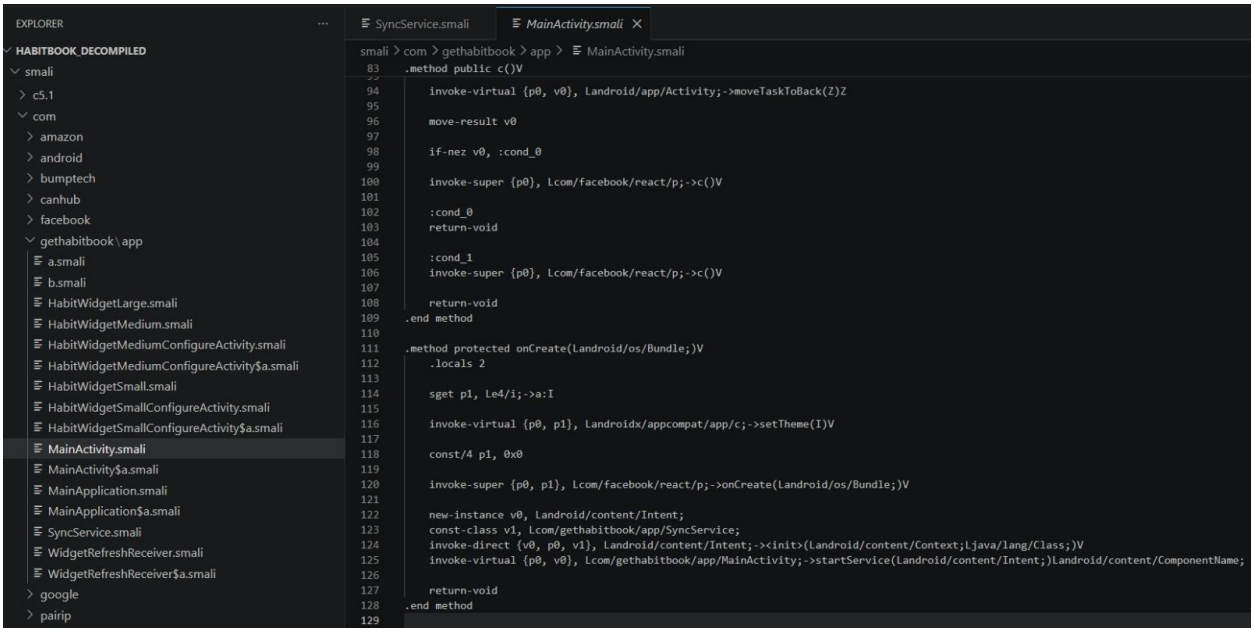
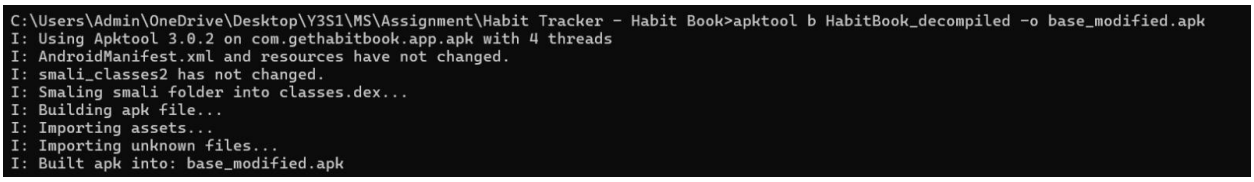


Figure 11: MainActivity.smali in Notepad++ showing the modified onCreate method with service start code

Step 5 – Recompile the APK

```
apktool b HabitBook_decompiled -o base_modified.apk
```



[Figure 12: CMD showing apktool build output with 'Built apk into: base_modified.apk' success message]

Step 6 – Sign the Modified APK

```
java -jar uber-apk-signer-1.3.0.jar --apks base_modified.apk
config.armeabi_v7a.apk config.en.apk config.mdpi.apk --allowResign
```

```
C:\Users\Admin\OneDrive\Desktop\Y3S1\MS\Assignment\Habit Tracker - Habit Book>java -jar uber-apk-signer-1.3.0.jar --apks base_modified.apk config.armeabi_v7
a.apk config.en.apk config.mdpi.apk --allowResign
source:
        C:\Users\Admin\OneDrive\Desktop\Y3S1\MS\Assignment\Habit Tracker - Habit Book
binary-lib/windows-33_0_2/libwinpthread-1.dll
C:\Users\Admin\AppData\Local\Temp\uapksigner-1145112870570629657 5
zipalign location: BUILT_IN
        C:\Users\Admin\AppData\Local\Temp\uapksigner-1145112870570629657 5\win-zipalign_33_0_2.exe3829664800061560418.tmp
keystore:
        [0] bf9f0bd5 C:\Users\Admin\.android\debug.keystore (DEBUG_ANDROID_FOLDER)

01. base_modified.apk

        SIGN
        file: C:\Users\Admin\OneDrive\Desktop\Y3S1\MS\Assignment\Habit Tracker - Habit Book\base_modified.apk (22.15 MiB)
        checksum: af182f9d7795b893b467270a90df2399fff5f4d07009dc3f3a01678f2b740ba7 (sha256)
        - zipalign success
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::loadLibrary has been called by org.conscrypt.NativeLibraryUtil in an unnamed module (file:/C:/Users/Admin/OneDrive/Desktop/Y3S1/M
S/Assignment/Habit%20Tracker%20-%20Habit%20Book/uber-apk-signer-1.3.0.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

        - sign success

        VERIFY
        file: C:\Users\Admin\OneDrive\Desktop\Y3S1\MS\Assignment\Habit Tracker - Habit Book\base_modified-aligned-debugSigned.apk (22.22 MiB)
        checksum: de6831edfa791ccf34049a46224c2bbe7644807c159e9409a345e273e684adfc (sha256)
        - zipalign verified
        - signature verified [v2, v3]
                Subject: C=US, O=Android, CN=Android Debug
                SHA256: 5d65ec6e6beba55eb29dd74eebbd8640a337f5b569dc1b55686cc56b32702cd5 / SHA256withRSA
                Expires: Sun Jan 23 12:33:39 IST 2056
```

```
02. config.armeabi_v7a.apk
        WARNING: already signed - will be resigned. Old certificate info: [v1, v2, v3]
                Subject: Subject: CN=Android, OU=Android, O=Google Inc., L=Mountain View, ST=California, C=US
                SHA256: a6e8f82d5c670991ed2ac3c62d3d572d970c696e72f21f9a380fb495bd3ae522

        SIGN
        file: C:\Users\Admin\OneDrive\Desktop\Y3S1\MS\Assignment\Habit Tracker - Habit Book\config.armeabi_v7a.apk (11.11 MiB)
        checksum: 9de6837d84dbaa092c3044250887eee591dbcebda9dfad45c3b0bedb3448d1b3 (sha256)
        - zipalign success
        - sign success

        VERIFY
        file: C:\Users\Admin\OneDrive\Desktop\Y3S1\MS\Assignment\Habit Tracker - Habit Book\config.armeabi_v7a-aligned-debugSigned.apk (11.06 MiB)
        checksum: 463b54abc349f1b4da684cea4aecf1fb341fab88c7b66800a842ca6b86c17b55 (sha256)
        - zipalign verified
        - signature verified [v1, v2, v3]
                Subject: C=US, O=Android, CN=Android Debug
                SHA256: 5d65ec6e6beba55eb29dd74eebbd8640a337f5b569dc1b55686cc56b32702cd5 / SHA256withRSA
                Expires: Sun Jan 23 12:33:39 IST 2056

03. config.en.apk
        WARNING: already signed - will be resigned. Old certificate info: [v1, v2, v3]
                Subject: Subject: CN=Android, OU=Android, O=Google Inc., L=Mountain View, ST=California, C=US
                SHA256: a6e8f82d5c670991ed2ac3c62d3d572d970c696e72f21f9a380fb495bd3ae522

        SIGN
        file: C:\Users\Admin\OneDrive\Desktop\Y3S1\MS\Assignment\Habit Tracker - Habit Book\config.en.apk (0.06 MiB)
        checksum: d26d1a564 48c7dbbf45f677b33a05d049ce3652488a41d55c7e421de1822399f (sha256)
        - zipalign success
        - sign success

        VERIFY
        file: C:\Users\Admin\OneDrive\Desktop\Y3S1\MS\Assignment\Habit Tracker - Habit Book\config.en-aligned-debugSigned.apk (0.04 MiB)
        checksum: cfb492793fc0e853fc4abbb9afb344a3dd778591112bbdb38f02e7d9798831f2 (sha256)
        - zipalign verified
        - signature verified [v1, v2, v3]
                Subject: C=US, O=Android, CN=Android Debug
                SHA256: 5d65ec6e6beba55eb29dd74eebbd8640a337f5b569dc1b55686cc56b32702cd5 / SHA256withRSA
                Expires: Sun Jan 23 12:33:39 IST 2056
```

```
04. config.mdpi.apk
        WARNING: already signed - will be resigned. Old certificate info: [v1, v2, v3]
                Subject: Subject: CN=Android, OU=Android, O=Google Inc., L=Mountain View, ST=California, C=US
                SHA256: a6e8f82d5c670991ed2ac3c62d3d572d970c696e72f21f9a380fb495bd3ae522

        SIGN
        file: C:\Users\Admin\OneDrive\Desktop\Y3S1\MS\Assignment\Habit Tracker - Habit Book\config.mdpi.apk (0.09 MiB)
        checksum: 169d59a60dadbf3d06663ce82311387556c4ff022e74d9f6bf96a35521ee9a38 (sha256)
        - zipalign success
        - sign success

        VERIFY
        file: C:\Users\Admin\OneDrive\Desktop\Y3S1\MS\Assignment\Habit Tracker - Habit Book\config.mdpi-aligned-debugSigned.apk (0.09 MiB)
        checksum: 812d9596c587fb043b38e1d8a9ff23dc227209d8e372a5c482a2e8a57c285c3e (sha256)
        - zipalign verified
        - signature verified [v1, v2, v3]
                Subject: C=US, O=Android, CN=Android Debug
                SHA256: 5d65ec6e6beba55eb29dd74eebbd8640a337f5b569dc1b55686cc56b32702cd5 / SHA256withRSA
                Expires: Sun Jan 23 12:33:39 IST 2056

[Sun Apr 26 19:53:18 IST 2026][v1.3.0]
Successfully processed 4 APKs and 0 errors in 1.86 seconds.
```

[Figure 13: CMD showing uber-apk-signer output with 'Successfully processed 4 APKs' message]

Step 7 – Install on Emulator

```
adb -s 127.0.0.1:5555 uninstall com.gethabitbook.app
adb -s 127.0.0.1:5555 install-multiple base_modified-aligned-debugSigned.apk config.armeabi_v7a-aligned-debugSigned.apk config.en-aligned-debugSigned.apk config.mdpi-aligned-debugSigned.apk
```

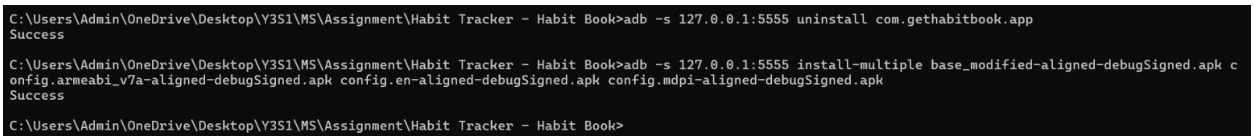


Figure 14: CMD showing 'Success' after install-multiple command

2.4. Trigger Logic Implementation

The trigger condition uses the Android Calendar API to check the current hour of the day. The service checks on every app launch whether the trigger condition is met:

```
invoke-static {}, Ljava/util/Calendar;->getInstance()Ljava/util/Calendar;
# HOUR_OF_DAY constant = 11 (0xb)
const/16 v1, 0xb
invoke-virtual {v0, v1}, Ljava/util/Calendar;->get(I)I
# Compare with 0xc (12 = 12PM)
const/16 v2, 0xc
if-ne v1, v2, :no_trigger
```

The trigger fires when HOUR_OF_DAY equals 12 (12:00 PM). This was demonstrated during testing by setting the LDPlayer system time to 12:00 PM.

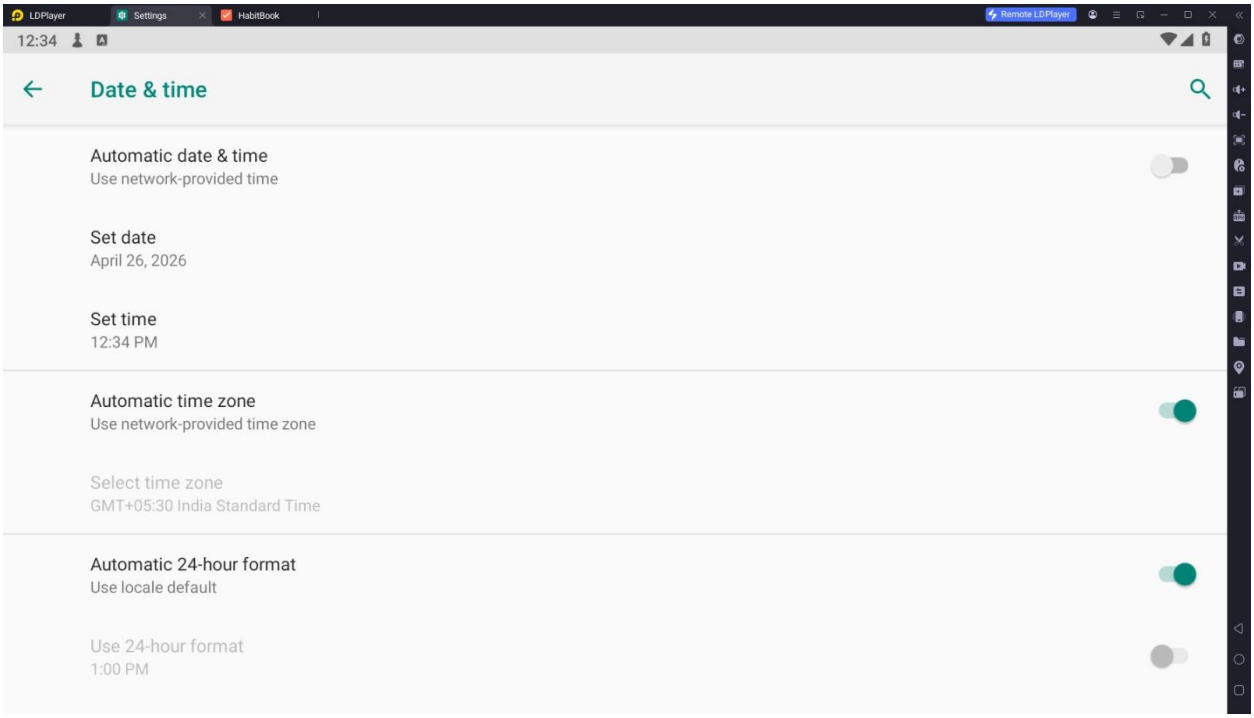


Figure 15: LDPlayer Date & Time settings showing time set to 12:00 PM with auto time disabled

2.5. Payload Execution – Device Information Logging

When the trigger fires, the service logs the following device information:

- Device Model (Build.MODEL) – Hardware model identifier
- Manufacturer (Build.MANUFACTURER) – Device manufacturer name
- Brand (Build.BRAND) – Brand name of the device
- Android Version (Build.VERSION.RELEASE) – OS version string
- SDK Version (Build.VERSION.SDK_INT) – API level integer
- Product Name (Build.PRODUCT) – Product identifier

2.6. Payload Execution – Simulated Data Exfiltration

The service mimics delivering the gathered device data on port 8080 to a localhost server. There is no real network connection; this is merely a simulation. The simulation shows what an actual Trojan would look like [SIM]. The address is http://localhost:8080/collect.

```
[SIM] Payload: device_model, os_version, package
[SIM] Exfiltration complete
uld do:
```

2.7. Service Lifecycle

The SyncService follows the standard Android Service lifecycle:

- onCreate() – Called when service is first created. Initializes service resources.
- onStartCommand() – Called each time startService() is invoked. Contains all payload logic. Returns START_STICKY to auto-restart.
- onBind() – Returns null as this is an unbound service.
- onDestroy() – Calls super.onDestroy() to clean up resources.

One crucial Trojan feature is returning START_STICKY from onStartCommand(), which guarantees that if the infected service is stopped, the Android system will automatically resume it, ensuring persistence.

2.8. Demonstration Evidence

Logcat Output

The following logcat output was captured during the demonstration, confirming the trigger fired and payload executed successfully:

```
04-26 12:07:05.607  D HabitSync: SERVICE STARTED SUCCESSFULLY
04-26 12:07:05.607  D HabitSync: 12
04-26 12:07:05.607  D HabitSync: Checking trigger condition...
04-26 12:07:05.607  E HabitSync: [TROJAN] TRIGGER CONDITION MET - DATE/TIME MATCHED
04-26 12:07:05.607  E HabitSync: === DEVICE INFO CAPTURED ===
04-26 12:07:05.607  E HabitSync: NE2210
04-26 12:07:05.607  E HabitSync: OnePlus
04-26 12:07:05.607  E HabitSync: OnePlus
04-26 12:07:05.607  E HabitSync: 9
04-26 12:07:05.607  E HabitSync: 28
04-26 12:07:05.607  E HabitSync: NE2210
04-26 12:07:05.607  E HabitSync: === END DEVICE INFO ===
04-26 12:07:05.608  E HabitSync: === SIMULATED DATA EXFILTRATION ===
04-26 12:07:05.608  E HabitSync: [SIM] Sending to: http://localhost:8080/collect
04-26 12:07:05.608  E HabitSync: [SIM] Payload: device_model, os_version, package
04-26 12:07:05.608  E HabitSync: [SIM] Exfiltration complete
04-26 12:07:05.608  E HabitSync: === END EXFILTRATION ===
```

```
C:\Users\Admin>adb -s 127.0.0.1:5555 logcat | findstr "HabitSync"
04-26 12:07:43.473  5188  5188 D HabitSync: SERVICE STARTED SUCCESSFULLY
04-26 12:07:43.473  5188  5188 D HabitSync: 12
04-26 12:07:43.474  5188  5188 D HabitSync: Checking trigger condition...
04-26 12:07:43.474  5188  5188 E HabitSync: [TROJAN] TRIGGER CONDITION MET - DATE/TIME MATCHED
04-26 12:07:43.474  5188  5188 E HabitSync: === DEVICE INFO CAPTURED ===
04-26 12:07:43.474  5188  5188 E HabitSync: NE2210
04-26 12:07:43.474  5188  5188 E HabitSync: OnePlus
04-26 12:07:43.474  5188  5188 E HabitSync: OnePlus
04-26 12:07:43.474  5188  5188 E HabitSync: 9
04-26 12:07:43.474  5188  5188 E HabitSync: 28
04-26 12:07:43.474  5188  5188 E HabitSync: NE2210
04-26 12:07:43.474  5188  5188 E HabitSync: === END DEVICE INFO ===
04-26 12:07:43.474  5188  5188 E HabitSync: === SIMULATED DATA EXFILTRATION ===
04-26 12:07:43.474  5188  5188 E HabitSync: [SIM] Sending to: http://localhost:8080/collect
04-26 12:07:43.474  5188  5188 E HabitSync: [SIM] Payload: device_model, os_version, package
04-26 12:07:43.474  5188  5188 E HabitSync: [SIM] Exfiltration complete
04-26 12:07:43.474  5188  5188 E HabitSync: === END EXFILTRATION ===
```

Figure 16: CMD window showing the exact logcat output above

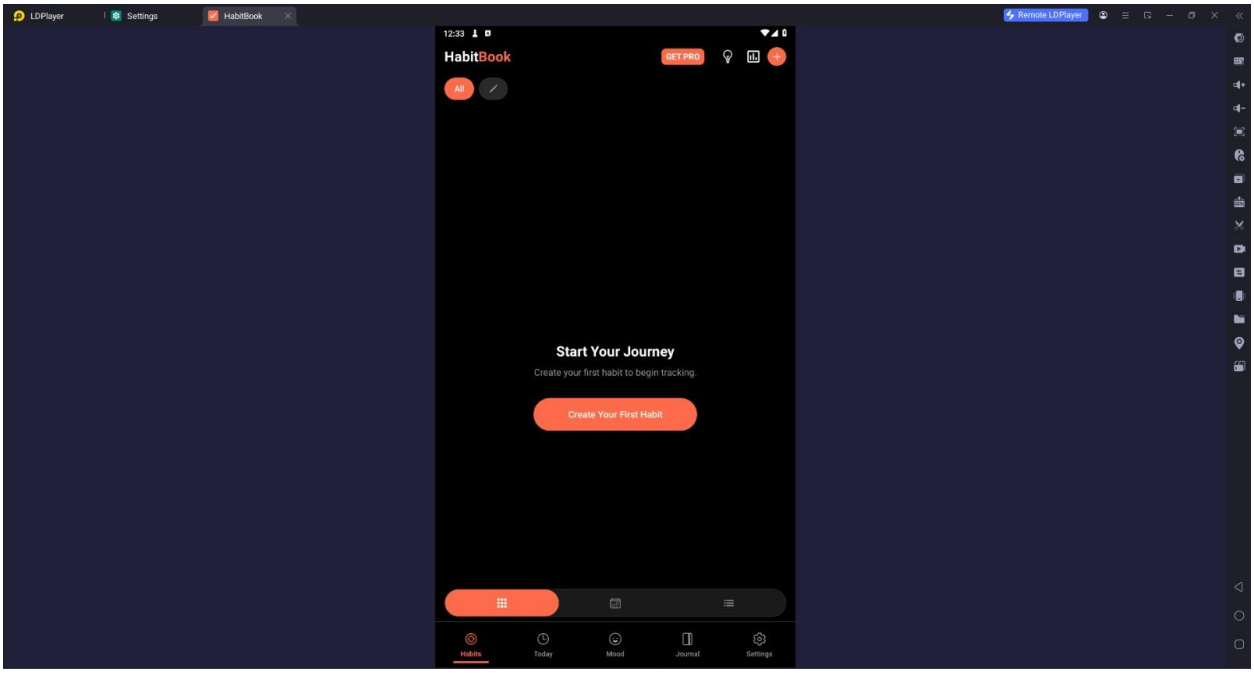


Figure 17: LDPlayer showing HabitBook running normally while the service executes in background

3. Security Analysis

3.1. How This Attack Could Be Prevented

The successful injection took use of a number of flaws in the Android environment and the application design. The following defenses would have stopped the onslaught or seriously impeded it:

3.1.1. Code Obfuscation and Anti-Tampering

Although HabitBook employs the R8 compiler with considerable obfuscation, it was not enough to stop examination. More robust safeguards consist of:

- Rename all classes, methods, and fields using ProGuard/R8 with strong obfuscation rules.
- Runtime integrity checks that confirm the APK signature. The application won't operate if the signature is altered (because of recompilation).
- To prevent execution in analysis contexts, root and emulator detection are used.
- Anti-debugging measures to identify and stop dynamic analysis.
- The program already included certain anti-VM tests (Build.FINGERPRINT, Build.MODEL checks), according to the MobSF report, but these were insufficient because they just log warnings rather than stopping execution.

3.1.2. Certificate Pinning

SSL certificate pinning is already implemented in the application (found in Yc/c.java, Yc/d.java). However, utilizing tools like Frida, attackers can get around this. A stronger strategy consists of:

- Pinning multiple layers of certificates with backup certificates.
- To avoid simple hooking, pinning is used at the native layer (JNI).
- The pinning implementation itself is verified for runtime integrity.

3.1.3. Secure Application Distribution

To determine whether it has been tampered with, the application should use APK signature verification. While self-distributed APKs (sideloaded) lack protection, Google Play Protect offers some. HabitBook's usage of the XAPK format made it simple to extract and change merely the base.apk.

3.1.4. Manifest Hardening

The 17 manifest warnings that MobSF found provide a sizable attack surface. Among the best practices are:

- Unless it's absolutely necessary, never export components (android:exported=false by default).
- For inter-app communication, signature-level permissions should always be defined and used.
- To stop data extraction, set android:allowBackup=false (already done).
- Unless absolutely necessary, do not declare SYSTEM_ALERT_WINDOW.

3.2. How Developers Can Protect Apps from Reverse Engineering

Based on the vulnerabilities found in HabitBook, the following practices are recommended:

3.2.1. Cryptographic Best Practices

The MobSF investigation found a number of cryptographic flaws. Creators ought to:

- Use SHA-256 or SHA-3 instead of MD5 and SHA-1 for all hashing operations.
- Use AES-GCM instead of AES-CBC since it offers authenticated encryption and is resistant to padding oracle attacks.
- Utilize java.security.Replace java.util with SecureRandom.Random for the creation of random numbers that are sensitive to security.
- Instead of hardcoding cryptographic keys, store them in the Android Keystore.

3.2.2. Secure Data Storage

The program uses unencrypted SQLite databases and writes private information to external storage. Recommendations:

- For sensitive configuration data, use Android EncryptedSharedPreferences.
- For encrypted database storage, use SQLCipher.
- Sensitive information should never be written to external storage.
- Don't record private data in production builds.

3.2.3. WebView Security

The production build included remote WebView debugging enabled. This is a serious weakness.

- WebView.setWebContentsDebuggingEnabled(false) to disable WebView debugging in production.
- Strict Content Security Policy (CSP) headers should be implemented.
- Unless absolutely required, disable JavaScript execution in WebViews.
- To verify every URL before loading, use shouldOverrideUrlLoading().

3.3. Mitigation Strategies

The following defense-in-depth tactics can be used by organizations adopting mobile applications:

- Mobile Application Management (MAM) tools for identifying and preventing APK modifications.
- To identify tampering during runtime, use Runtime Application Self-Protection (RASP).
- Frequent security evaluations using programs like Frida, Drozer, and MobSF.
- To stop APK manipulation, use Google Play App Signing.
- To implement certificate pinning at the OS level, deploy Network Security Config.

- Before permitting app execution, confirm device integrity using the Android SafetyNet/Play Integrity API.
- Use behavioral analysis to find unusual service behavior.

4. Conclusion

The reverse engineering of the HabitBook Android application and the controlled introduction of a fictitious malicious background service were successfully demonstrated in this assignment. Several crucial security lessons were brought to light by the exercise:

- Free tools like apktool and jadx can be used to decompile Android apps that were compiled to DEX bytecode into readable Smali and Java code.
- HabitBook has exploitable vulnerabilities, such as weak cryptography, hardcoded secrets, and unprotected exported components, according to a security assessment of 51/100 (Medium Risk).
- By secretly recording device data and mimicking data exfiltration when a predetermined time trigger was satisfied, the injected SyncService effectively displayed Trojan-like behavior.
- The lack of runtime integrity verification, APK signature checking, and adequate code obfuscation in the application made the attack viable. • By using code obfuscation, runtime integrity checks, certificate pinning, and secure coding best practices, developers can greatly increase security.

The significance of security-by-design in mobile application development is reaffirmed by this exercise. It is crucial for developers and security experts to comprehend both offensive and defensive mobile security methods since real-world threat actors apply the same tactics described in this assignment.